DSA Sheet

DOCUMENT 3 OF 3 · OPTIONAL (BONUS)

API Design, Scalability Improvements & Trade-offs

The REST API contract, measured capacity with a roadmap for further scale, and the reasoning behind each major design decision.

Project	DSA Sheet: plan, track and revise Data Structures & Algorithms (MERN + TypeScript)
Prepared for	Apna College · Technical Team Lead assessment
Author	MONUPRAJAPAT
Date	20 October 2026
Live application	https://dd2dekdh8uhzz.cloudfront.net
Source code	https://github.com/MONUPRAJAPAT/DSA-Sheet

Covers: **Part A** API design (conventions, error model, every endpoint with examples) · **Part B** scalability: measured load-test results and future improvements · **Part C** trade-offs: system-level decisions, schema decisions and Architecture Decision Records.

Contents

Part A — API design

- Conventions
- Endpoints
- Design decisions

Part B — Scalability improvements

- Measured: load-test results
- Roadmap: future scalability improvements

Part C — Trade-offs in design decisions

- System-level trade-offs
- Database schema trade-offs
- Architecture Decision Records

Part A — API design

REST over HTTPS, JSON, versioned under `/api/v1`. The interactive OpenAPI 3.1 docs are served by the API itself at `/api/docs` (raw spec at `/api/openapi.json`). Request schemas in the docs come from the same Zod schemas the server validates with, so the docs cannot drift from the code.

Conventions

Concern	Convention
Envelope	Success: `{ "success": true, "data": … }` · Error: `{ "success": false, "error": { "code", "message", "details?" } }`
Auth	`Authorization: Bearer <accessToken>` (JWT, 15 min). The refresh token is an httpOnly cookie scoped to `/api/v1/auth`
Timezone	Clients send `X-Timezone: Asia/Kolkata` (IANA). Streaks, "today", revisions and the calendar use the student's local day
Validation	Every body, query and path parameter is validated with Zod. A failure returns `400 VALIDATION_ERROR` listing **all** issues
Idempotency	`PUT /progress/:problemId` is an upsert: repeating it never creates duplicates
Caching	Private responses send `Cache-Control: no-store`. The public sheet sends `ETag` + `Cache-Control: public, max-age=60, stale-while-revalidate=300` and answers `304` on `If-None-Match`
Rate limits	300 req/min/IP across the API. Login: 10 **failed** attempts / 15 min per IP + email (successes don't count). Register: 10 / hour per IP. Demo: 60 / 15 min per IP. `429 RATE_LIMITED` with `RateLimit-*` headers and a message saying when to retry
Tracing	Every response carries `X-Request-Id` (also in the structured logs)

Error codes

HTTP	`code`	When
400	`VALIDATION_ERROR` / `BAD_REQUEST`	Invalid input, malformed JSON, invalid id
401	`UNAUTHORIZED` / `TOKEN_EXPIRED` / `INVALID_CREDENTIALS`	Missing or expired token, wrong password
403	`FORBIDDEN`	Role not allowed
404	`NOT_FOUND`	Unknown route, or a resource that doesn't exist **or belongs to another user** (no information leak)
409	`CONFLICT`	Email already registered
429	`RATE_LIMITED`	Too many requests
500	`INTERNAL_ERROR`	Unexpected error (details hidden in production, logged with the request id)

```
{
  "success": false,
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Password must contain a number",
    "details": [
      { "path": "body.email", "message": "Enter a valid email" },
      { "path": "body.password", "message": "Password must contain a number" }
    ]
  }
}
```

Endpoints

Auth: `/api/v1/auth`

Method	Path	Auth	Description
POST	`/register`	—	Create an account → `201 { user, accessToken }` + sets refresh cookie
POST	`/login`	—	Log in → `{ user, accessToken }` + sets refresh cookie
POST	`/demo`	—	Log in as the seeded demo student (for evaluators)
POST	`/refresh`	cookie	**Rotates** the refresh token and returns a new access token
POST	`/logout`	cookie	Revokes this device's session and clears the cookie
POST	`/logout-all`	Bearer	Revokes every session of the user
GET	`/me`	Bearer	Current user

```
POST /api/v1/auth/login
Content-Type: application/json

{ "email": "asha@example.com", "password": "Passw0rd!" }
```

```
HTTP/1.1 200 OK
Set-Cookie: dsa_rt=<opaque>; Path=/api/v1/auth; Expires=…; HttpOnly; Secure; SameSite=Strict

{
  "success": true,
  "data": {
    "user": {
      "id": "6ad6…a1", "name": "Asha", "email": "asha@example.com", "role": "student",
      "preferences": { "dailyGoal": 3, "studyDays": [1,2,3,4,5,6], "targetDate": null },
      "lastActive": { "problemId": null, "at": null },
      "createdAt": "2026-10-20T10:00:00.000Z"
    },
    "accessToken": "eyJhbGciOiJIUzI1NiIs…"
  }
}
```

Sheet: `/api/v1/sheet` (public, cacheable)

Method	Path	Description
GET	`/`	The whole hierarchy: `chapters[] → topics[] → problems[]`, plus totals and a content `version`

```
{
  "success": true,
  "data": {
    "version": "b0a0b5440596",
    "totals": { "problems": 200, "easy": 55, "medium": 120, "hard": 25 },
    "chapters": [
      {
        "id": "…",
        "title": "Arrays",
        "slug": "arrays",
        "order": 1,
        "icon": "brackets",
        "problemCount": 21,
        "description": "The foundation of DSA — …",
        "topics": [
          {
            "id": "…",
            "title": "Prefix Sum",
            "slug": "prefix-sum",
            "difficulty": "Medium",
            "order": 2,
            "description": "Pre-compute running totals to answer range queries in O(1).",
            "resources": {
              "youtube": "https://www.youtube.com/watch?v=…",
              "practice": {
                "platform": "LeetCode",
                "url": "https://leetcode.com/problem-list/prefix-sum/"
              },
              "article": "https://www.geeksforgeeks.org/dsa/prefix-sum-array-implementation-
applications-competitive-programming/"
            },
            "problems": [
              {
                "id": "…",
                "title": "Subarray Sum Equals K",
                "slug": "subarray-sum-equals-k",
                "difficulty": "Medium",
                "order": 4,
                "tags": ["array", "hash-table", "prefix-sum"],
                "resources": {
                  "youtube": "…",
                  "practice": {
                    "platform": "LeetCode",
                    "url": "https://leetcode.com/problems/subarray-sum-equals-k/"
                  },
                  "article": "…"
                }
              }
            ]
          }
        ]
      }
    ]
  }
}
```


Content is identical for every student, so it is separated from per-user data. That lets CloudFront and browsers cache it, which removes the largest payload (~100 KB) from the API's hot path.

Progress: `/api/v1/progress` (Bearer)

Method	Path	Description
GET	`/`	All of the student's progress rows (`problemId, status, starred, notes, solvedAt, updatedAt`)
PUT	`/:problemId`	**Idempotent upsert** of `{ status?, starred?, notes? }` (at least one field)
GET	`/stats`	Dashboard: solved/total, per difficulty, per chapter, streak, revisions due, `continue` (resume target)
GET	`/heatmap`	`[{ date, count }]` solved per local day, last 365 days
POST	`/last-active`	`{ problemId }`: remember the last opened problem
DELETE	`/`	Reset all progress (and pending spaced-repetition revisions)

```
PUT /api/v1/progress/6ad66c29342fdea543b150ef
Authorization: Bearer …
X-Timezone: Asia/Kolkata

{ "status": "solved" }
```

Side effects, in the same request:

1. `lastActive` is updated, so the next login resumes here.
2. Pending **study** tasks for this problem are marked done.
3. **Spaced-repetition** revisions are scheduled at +1, +3, +7 and +21 days.

Un-ticking removes the pending revisions.

```
GET /api/v1/progress/stats →
{
  "solved": 34, "total": 200,
  "byDifficulty": { "Easy": { "solved": 16, "total": 55 }, "Medium": { "solved": 16, "total": 120
}, "Hard": { "solved": 2, "total": 25 } },
  "byChapter": [{ "chapterId": "…", "solved": 21, "total": 21 }],
  "streak": { "current": 16, "longest": 16 },
  "revisionsDue": 7,
  "continue": { "problemId": "…", "title": "Intersection of Two Arrays", "chapterTitle":
"Hashing", "topicTitle": "Hash Map & Set Basics" }
}
```

Calendar: `/api/v1/calendar` (Bearer)

Method	Path	Description
GET	`/events?from=YYYY-MM-DD&to=YYYY-MM-DD`	Events in a range (month/week view). Each event includes `overdue` and the linked `problem`
POST	`/events`	Create `{ date, type?, problemId?, title?, startTime?, endTime?, notes? }`
PATCH	`/events/:id`	Reschedule (drag and drop), edit, or complete `{ status: "done" }`. Completing a study task also ticks the problem

Method	Path	Description
DELETE	`/events/:id`	Delete
GET	`/agenda`	Today's tasks + every overdue pending task
POST	`/rollover`	Move overdue tasks to today; originals are kept as `missed` (history)
GET	`/plan`	Active auto study plan (404 if none)
POST	`/plan`	Generate plan `{ startDate, targetDate, dailyGoal, studyDays[], chapterIds?, replaceExisting? }`
DELETE	`/plan`	Remove the plan and its pending tasks
GET	`/export.ics`	RFC 5545 iCalendar download (Google / Apple / Outlook)

```
POST /api/v1/calendar/plan
{ "startDate": "2026-10-20", "targetDate": "2026-12-15", "dailyGoal": 3, "studyDays":
[1,2,3,4,5,6] }

201 →
{ "created": 147, "problemsScheduled": 147, "firstDate": "2026-10-20", "lastDate": "2026-12-15",
  "fitsBeforeTarget": false, "unscheduled": 19, "suggestedDailyGoal": 4 }
```

Users: `/api/v1/users` (Bearer)

Method	Path	Description
PATCH	`/me`	Update `name`, `dailyGoal`, `studyDays`, `targetDate`

Health

Method	Path	Description
GET	`/api/v1/health`	`200 { status: "ok", db: "up" }` or `503` when the DB is unreachable. Used by the ALB health check

Design decisions

- **Resource-oriented REST, not RPC.** Progress is addressed by problem (`PUT /progress/:problemId`), so the client never needs to know a progress row's id, and retries are safe.
- **Public vs. private split** (`/sheet` vs `/progress`). Public data can be cached at the edge, and private data is never cached.
- **Server-side side effects** (revisions, study-task sync) keep every client consistent; the web UI is not the only possible client.
- **404 instead of 403 for other users' resources**, so ids cannot be probed.
- **Versioned prefix** (`/api/v1`) leaves room for breaking changes without breaking deployed clients.

Part B — Scalability improvements

How the current design scales (see Document 1, section 7) has been measured, and the roadmap below shows what to add as usage grows.

Measured: load-test results

Tool: k6 (`k6.js`) · **Target:** one production-build API process (`node dist/server.js`, single Node.js event loop, no clustering) + MongoDB 8.2 in Docker · **Machine:** a developer laptop that was also running MongoDB, Docker and k6 itself, so these numbers are a conservative floor · **Date:** 2026-10-20

1. Realistic student sessions: 500 concurrent students

Each virtual student repeatedly: opens the sheet → loads progress → ticks/unticks a random problem (write) → views dashboard stats → opens the month calendar, with 1–3 s think time between actions.

Metric	Result
Peak concurrent students	**500**
Requests	24,687 (0 failed)
Checks passed	**100 %** (24,685 / 24,685)
Latency (all)	median **5.5 ms** · p95 **28 ms** · p99 **77 ms**
Reads p95 / Writes p95	28 ms / 30 ms
Thresholds (p95 read < 300 ms, write < 500 ms, errors < 1 %)	✅ all passed

2. Capacity: no think time (sheet excluded, served by CloudFront)

Weighted mix: 30 % `GET /progress`, 30 % `PUT /progress/:id`, 20 % `GET /progress/stats`, 20 % `GET /calendar/agenda`.

Load	Throughput	Median	p95	p99	Errors
Constant **600 req/s** for 45 s	595 req/s	**2.3 ms**	105 ms	301 ms	**0 %**
Ramp 100 → 1,600 req/s	saturates at **~800–900 req/s**	5.6 ms	3.2 s (queueing)	4.0 s	**0 %**

Takeaway: a single API process handles ~600 req/s comfortably. Past saturation it degrades gracefully (requests queue, nothing fails). Because the API is stateless, capacity scales linearly with ECS tasks behind the load balancer.

Reproduce

```
npm run build -w server
API_RATE_LIMIT=1000000 LOG_LEVEL=warn NODE_ENV=production node --env-file=server/.env
server/dist/server.js &
docker run --rm -i -e BASE_URL=http://host.docker.internal:4000 grafana/k6 run - < loadtest/k6.js
# realistic
docker run --rm -i -e MODE=stress -e BASE_URL=http://host.docker.internal:4000 grafana/k6 run - <
loadtest/k6.js # capacity
```

Roadmap: future scalability improvements

1. **Redis (ElastiCache)** for the shared rate-limit store and the sheet cache, plus a pub/sub cache-invalidation signal when admins edit content.
2. **Read replicas** (`readPreference: secondaryPreferred`) for dashboard/heatmap aggregations if reads dominate.
3. **Sharding** `userProgress` and `calendarEvents` on hashed `userId` beyond a few hundred million rows. Every per-user query already includes `userId` , so it would target a single shard.
4. **Event-driven side effects:** move revision scheduling and analytics to a queue (SQS) so the write path is a single upsert.
5. **WAF + bot control** on CloudFront; private networking to Atlas (PrivateLink, NAT fixed egress).
6. **Precomputed leaderboards / cohort analytics** in a separate analytical store if product needs grow.
7. **PWA offline mode:** queue ticks offline and sync on reconnect (the idempotent PUT makes replay safe).

Part C — Trade-offs in design decisions

System-level trade-offs

Decision	Chosen	Alternative	Reasoning
Database	**MongoDB**	PostgreSQL	The content is naturally hierarchical documents (topic → resources), progress is a simple keyed upsert, and the brief asks for MERN. Postgres would also work; there are no multi-entity transactions on the hot path
Progress model	**One doc per (user, problem)**	Array inside the user document	Avoids unbounded documents and write contention; indexed and idempotent (ADR-002)
Auth	**JWT access + rotating refresh cookie**	Server sessions only / JWT in localStorage	Stateless scaling *and* revocation *and* XSS safety (ADR-003)
Stats	**Computed on read**	Counters maintained on write	Always consistent, simpler; ~1 ms per user at this size. Revisit only if profiling demands it
Topology	**Single origin via CloudFront** (`/api/*` → ALB)	Separate API domain + CORS	First-party cookies, no CORS preflights, one TLS certificate (ADR-004)
Compute	**ECS Fargate**	EC2 + PM2 / Lambda	No servers to patch, autoscaling and rolling deploys built in. Lambda cold starts and connection pooling to MongoDB are awkward
Live demo hosting	**Lite profile (single Graviton EC2 behind CloudFront)**	The production ALB + Fargate stack	About $12 instead of $82 a month, so the demo runs for the whole Free plan period. Same image, secrets and routing; the production template stays in the repo for real scale.
Calendar dates	**User-local `YYYY-MM-DD` strings**	UTC timestamps	Correct "today" for every timezone; trivial range queries (ADR-005)
Monorepo + shared package	**npm workspaces**	Separate repos	One PR changes API + UI + types atomically; validation rules can't diverge
Content links	**Fetched and verified from the platforms**	Hand-typed URLs	Official titles and difficulty; no dead or mismatched videos; re-validated weekly in CI

Database schema trade-offs

#	Decision	Alternative	Why this choice
T1	**Separate `userProgress` collection** (one doc per user-problem)	Embed a `progress[]` array in `users`	Embedded arrays grow without bound (16 MB document limit), every tick rewrites a growing document, and concurrent updates contend on one document. A junction collection gives indexed, idempotent, O(1) upserts and easy aggregation
T2	**Denormalize `chapterId` + `difficulty` into progress rows**	`$lookup` problems on every dashboard load	Values are immutable for a problem; copying them once on insert makes stats a single indexed scan of the user's rows

#	Decision	Alternative	Why this choice
T3	**Compute stats on read** (≤ 200 rows/user, indexed)	Maintain counters on the user document	Always consistent, with no dual-write bugs. At ~1 ms per query it is cheaper than the complexity of counters. Counters can be added later if profiling shows a need
T4	**Calendar dates as `YYYY-MM-DD` strings in the user's timezone**	UTC `Date`	"What day is it for this student?" is the business question. Strings avoid every off-by-one-day bug around midnight IST, sort lexicographically, and make range queries trivial. The client sends `X-Timezone`
T5	**Upsert content by `slug` in the seed**	Drop and re-insert	`_id`s never change, so students' progress survives content updates. The seed is idempotent and safe to run on every deploy
T6	**Store only a SHA-256 hash of refresh tokens**	Store raw tokens / stateless refresh JWTs	A database leak does not leak usable sessions, and server-side rows still allow logout and revocation
T7	**Lazy progress rows** (absence = todo)	Pre-create 200 rows per user at sign-up	Zero write amplification on registration, and storage grows only with real activity
T8	**Spaced-repetition revisions stored as calendar events**	A separate `revisions` collection	One timeline for everything the student plans, tracks and revisits. Revisions appear on the calendar with no special-casing

Architecture Decision Records

ADR-001: TypeScript monorepo with a shared contract package

Status: Accepted · **Date:** 2026-10-20

Context

The client and server must agree on request shapes, validation rules (password policy, date formats) and response types. In separate repos these drift, and bugs surface only at runtime.

Decision

- One repository with npm workspaces: `shared/`, `server/`, `client/`.
- `shared` exports **Zod schemas, constants and DTO types**:
  - the server validates every request with these schemas;
  - the client uses the same schemas for form validation and the inferred types for API calls;
  - the OpenAPI docs are generated from them.
- TypeScript runs in `strict` mode with `noUncheckedIndexedAccess` everywhere.

Consequences

- ✅ A breaking API change fails `npm run typecheck` in CI before it can ship.
- ✅ One PR can change the API, UI and docs together.
- ➖ The build has to bundle `shared` into the server (`tsup noExternal`); this is handled in the build config.

ADR-002: Store progress as one document per (user, problem)

Status: Accepted · **Date:** 2026-10-20

Context

Progress is the hottest data in the system: every checkbox click writes it, and every page reads it. Options:

1. an embedded `progress[]` array on the user document;
2. a map `{ problemId: status }` on the user document;
3. a separate `userProgress` collection.

Decision

Option 3: a `userProgress` collection with a **unique compound index** `{ userId, problemId }`, written with an **upsert**. A missing row means "todo". `chapterId` and `difficulty` are copied in on insert.

Consequences

- ✅ Writes are O(1), touch one small document, and are idempotent. Retries and double-clicks are safe.
- ✅ No unbounded documents (16 MB limit) and no write contention on a single hot user document.
- ✅ Aggregations (per difficulty, per chapter, streak) are single indexed scans, with no `$lookup`.
- ✅ `userId` is a natural future shard key.
- ➖ Loading progress means reading up to 200 small documents instead of one. That is measured at a few milliseconds and irrelevant at this size.

ADR-003: Short-lived JWT access token + rotating httpOnly refresh cookie

Status: Accepted · **Date:** 2026-10-20

Context

We need secure login that scales horizontally, supports logout and revocation, and is resistant to XSS and CSRF.

Options

Option	Problem
JWT in `localStorage`	Readable by any injected script (XSS = account takeover); can't be revoked
Server sessions only	Needs a shared session store (Redis) and a DB/cache hit on every request
Access JWT in memory + refresh token in an httpOnly cookie	—

Decision

- The access JWT lasts 15 minutes. It is held in memory and sent as a Bearer header, and verified statelessly.
- The refresh token is 384 random bits:

  - it lives in an `httpOnly; Secure; SameSite=Strict; Path=/api/v1/auth` cookie;
  - it is stored server-side only as a SHA-256 hash, in `sessions`, with a TTL index;
  - it is **rotated on every use**.
- The client refreshes silently on a 401, single-flighted so parallel requests trigger only one refresh.

Consequences

- ✅ No auth DB hit on normal requests, so any API instance can serve any user.
- ✅ Logout and "log out everywhere" take effect immediately; a leaked DB contains no usable tokens.
- ✅ CSRF can't use the cookie (strict same-site, auth-path only, and data endpoints need the header).
- ➖ An access token remains valid for up to 15 minutes after revocation. Accepted for this domain.

ADR-004: Serve web and API from one CloudFront origin

Status: Accepted · **Date:** 2026-10-20

Context

Hosting the SPA (S3) and API (ALB) on different domains requires CORS, preflight requests on every non-simple call, and `SameSite=None` cookies, which are weaker and blocked by some browsers' third-party cookie rules.

Decision

- A single CloudFront distribution routes `/*` to S3 (with a SPA rewrite function) and `/api/*` to the ALB.
- The ALB only accepts CloudFront's origin-facing IP ranges carrying a secret `X-Origin-Verify` header.
- Locally, the Vite dev proxy (and nginx in `docker-compose`) reproduce the same topology.

Consequences

- ✅ First-party `SameSite=Strict` cookies, no CORS, no preflights, one TLS certificate.
- ✅ The public sheet endpoint is cached at the edge, close to students.
- ✅ The API cannot be reached by bypassing the CDN.
- ➖ API cache behaviour must be configured carefully: a custom cache policy with TTL 0 by default, `Authorization` included in the key, and `no-store` on private responses.

ADR-005: Calendar days as user-local `YYYY-MM-DD` keys

Status: Accepted · **Date:** 2026-10-20

Context

"What's due today?", "Did I keep my streak?" and "Schedule a revision in 3 days" are questions about the **student's local day**. Storing UTC instants and converting everywhere causes off-by-one-day bugs, for example a problem solved at 00:30 IST counting as the previous day.

Decision

- Calendar events store `date` as a `YYYY-MM-DD` string in the student's timezone, plus optional `HH:mm` times.

- The client sends `X-Timezone` (IANA) on every request; the server validates it and uses it to compute "today".
- Streak and heatmap aggregations use MongoDB's `$dateToString` with that timezone.
- Instants that are genuinely instants (`solvedAt`, `createdAt`) stay UTC `Date`s.

Consequences

- ✅ Correct day boundaries for every student; trivial, indexable range queries (`date ⩾ from && date ⩽ to`).
- ✅ Matches how calendars and iCalendar all-day events work.
- ➖ If a student changes timezone, previously planned days keep their calendar date. That is the expected calendar behaviour.